

Obtain an X.509 Certificate

- Requires both a Public (.crt ,.cer) and Private Key (.p12,.pfx).
- See script below for certificate requirements.
- Edit the DNSName and Subject
- Edit the NotBefore and NotAfter values
- The .CER and .PFX file will be in your Documents folder.
- We create the certificate in the CurrentUser store due to permissions issues. It gets exported and then imported to the LocalMachine store. If we are testing this script and not running it from Intune (in the SYSTEM context), we need to grant ourselves access to it.

```
$params = @{
    Type = 'Custom'
    DnsName = 'ianbaxter.onmicrosoft.com'
    CertStoreLocation = 'Cert:\CurrentUser\My'
    KeyExportPolicy = 'Exportable'
    KeySpec = 'Signature'
    Subject = 'CN=Data Collect'
    KeyUsage = "DigitalSignature"
    KeyAlgorithm = 'RSA'
    KeyLength = 2048
    HashAlgorithm = 'SHA256'
    NotBefore = (Get-Date)
    NotAfter = (Get-Date).AddMonths(24)
}

#
# Create the Self-Signed Certificate
New-SelfSignedCertificate @params
# Locate and load the certificate we just created
$cert = Get-ChildItem -Path "Cert:\CurrentUser\My" | Where-Object {$_.Subject -Match "Data Collect"}
# Get thumbprint
$thumbprint = ($cert | Select-Object Thumbprint, FriendlyName).Thumbprint
#
# Export the Public Key (.CER)
Export-Certificate -Cert $cert -FilePath "$env:USERPROFILE\Documents\Data_Collect.cer"
#
# Create the password we need for the Private Key
# Enter password here
$MyPassword = "SomePassword"
$mypwd = ConvertTo-SecureString -String $MyPassword -Force -AsPlainText
#
# Export the Private Key (.PFX)
Export-PfxCertificate -Cert $cert -FilePath "$env:USERPROFILE\Documents\Data_Collect.pfx" -Password $mypwd
#
# Delete by ThumbPrint
Remove-Item -Path "Cert:\CurrentUser\My\$thumbprint" -DeleteKey
```

Create and secure an App Registration

Launch the Entra ID portal

Choose “App Registrations” from the left menu bar.

Click “New Registration” from the top menu bar.

Give your Service Principal a name.

Unless you know you need it, you should probably just allow “Single Tenant Only” for the Account Types.

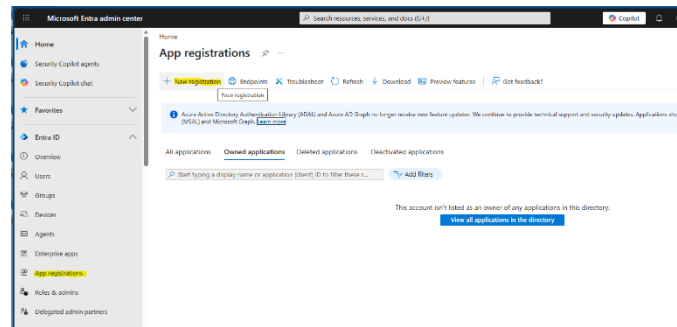
You should make a note of the following properties:

- Application (client) ID
- Directory (tenant) ID

These will be needed to test the Custom Compliance script.

Select “Certificates and Secrets” from the left menu bar.

Select the “Certificates” tab and click the “Upload Certificate” button.



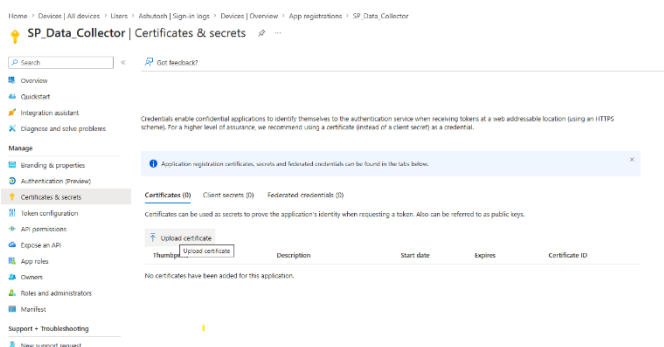
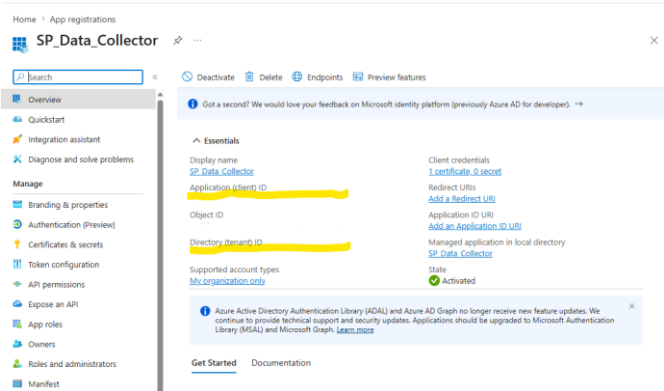
Register an application

* Name
The user-facing display name for this application (this can be changed later).

Supported account types
Choose the account types that can use this application or access this API.
 [Help me choose](#)

Redirect URI (optional)
We'll return the authentication response to this URI after successfully authenticating the user. Providing this now is optional and it can be changed later, but a value is required for most authentication scenarios.

Register an app you're working on here. Integrate gallery apps and other apps from outside your organization by adding from [Enterprise applications](#).

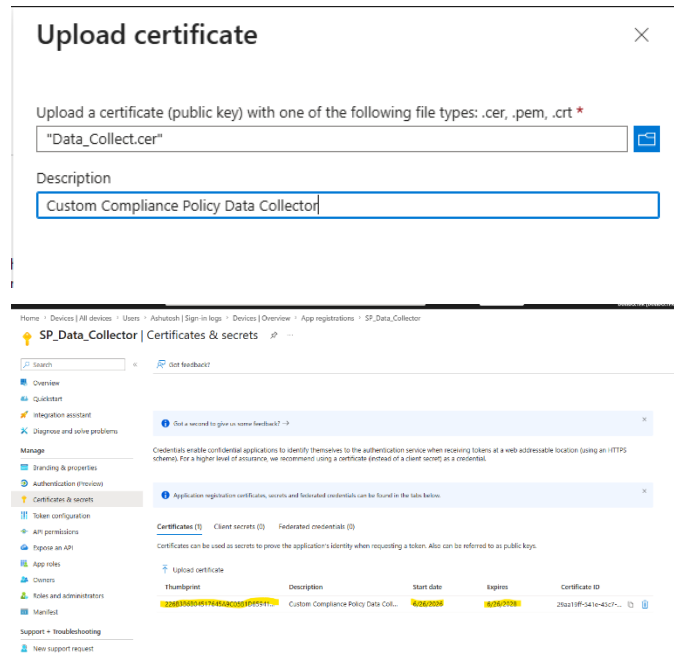


Select the file containing the public key (.crt or .cer).

Give the Descriptor some data to properly identify the certificate.

Make a note of the thumbprint so you can verify the private key when you install it on the test device.

Verify that you have the right dates on the certificate. (Note: You will see an icon to the right of the certificate if it has expired.)

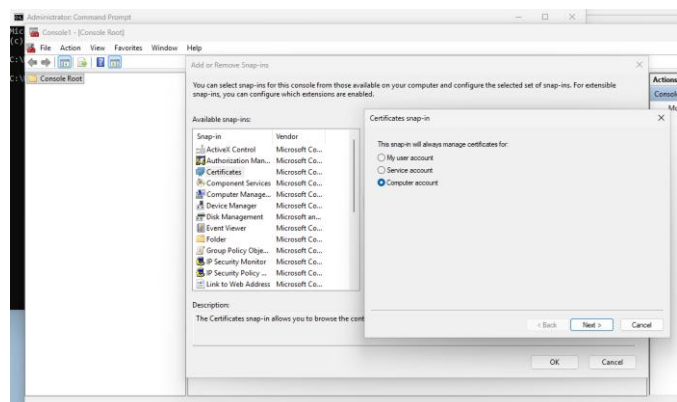


Install the Private Key on the test machine

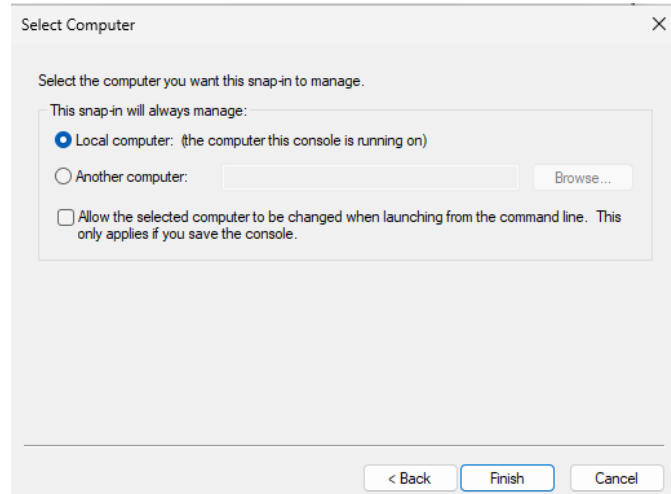
Since this is a test machine, you will be running the script as a user. The LocalMachine certificate store restricts users from reading the private key information from the certificate.

You need to have administrative rights to perform this step. Use the Windows search to find the icon for “CMD”. Right-click on the icon and choose “Run as Administrator”. Using the command prompt, run “MMC.EXE”, Select “File”, “Add/Remove Snapin...” and add the “Certificates” snapin.

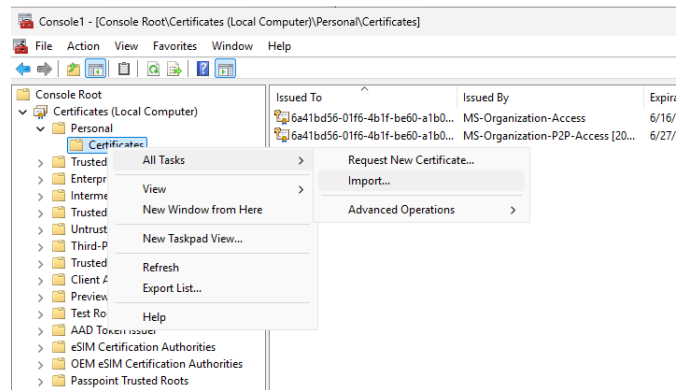
Select “Computer Account” and click “Next.”



Select “Local Computer” and click “Finish”.



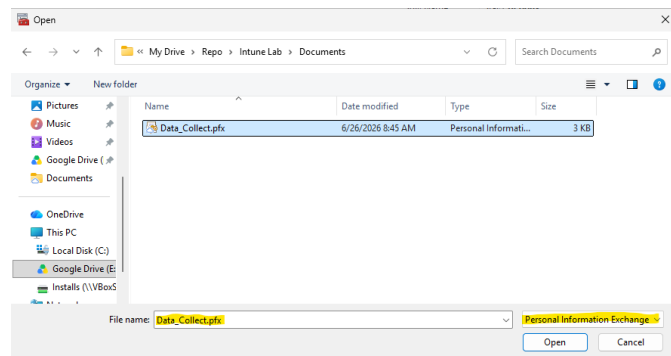
Locate the \Personal\Certificates folder in the left pane. Right-click, chose “All Tasks” and “Import...”



Set the file type dropdown to “Personal Information Exchange”.

Browse to your private key file and select it.

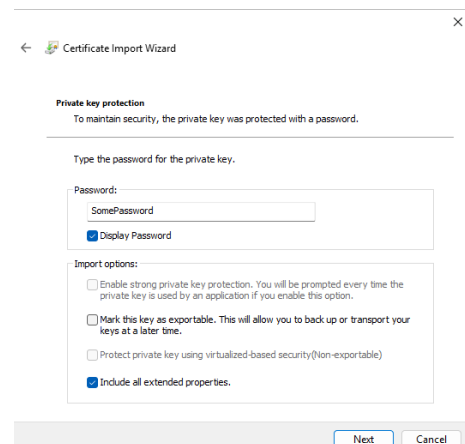
Click “Open”.



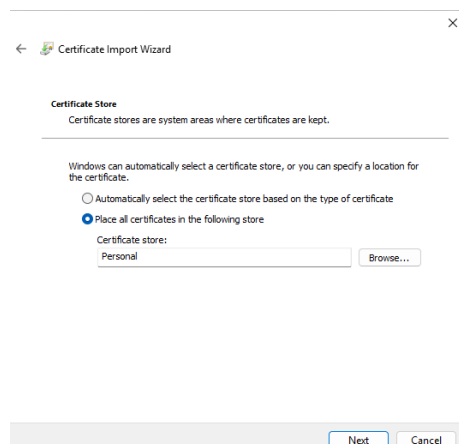
Remember the password you used to create the private key file? Enter it in the password text box.

If you don’t have the password, you need to get a new certificate and start over.

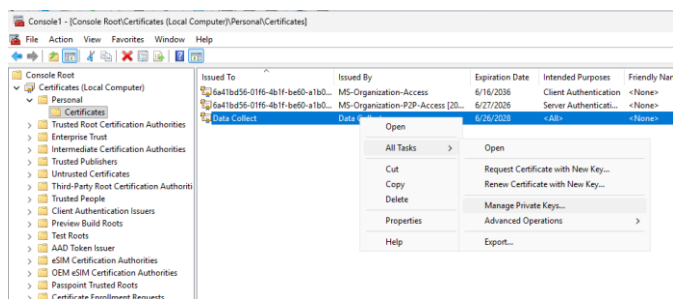
Do not select the option to make the certificate exportable.



Verify the certificate store is “Personal” and click “Next”.



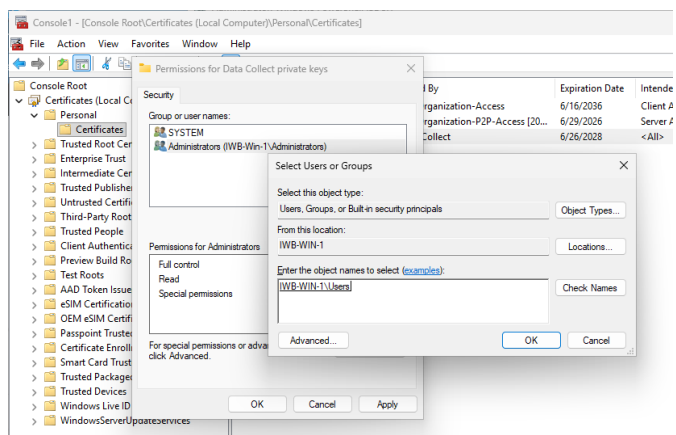
Right-click on the certificate, choose “All Tasks” and “Manage Private Keys...”.



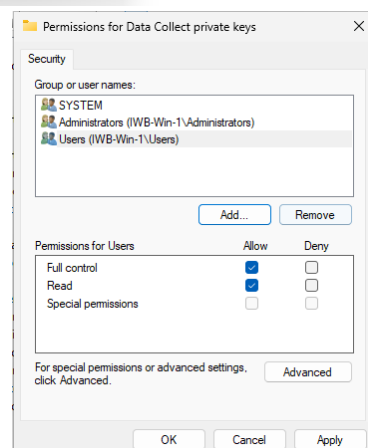
A permissions dialog will pop up. Click “Add”.

Enter “Users” in the bottom box, click “Check Names” to ensure the group or account can be resolved.

Click “OK”.



Verify the group or account has at least “Read” and “Full Control” access. You need at least “Read”, but “Full Control” will be needed to delete the certificate later.



Alter the Inventory Script and test (save JSON)

We are going to use a Windows PowerShell script for testing (Custom_Inventory.ps1). You will need to run the script using administrative access.

Enter “Powershell_ise” in the Windows Search box. Locate the icon for “Windows PowerShell ISE 5.1”, right-click on it and choose “Run as Administrator”.

You will need to run “Set-ExecutionPolicy -Scope LocalMachine -ExecutionPolicy Unrestricted” in the console.

The first few lines of the script are configuration variables.

Populate:

- \$ApplicationID
- \$TenantID
- \$CertName
- \$LogFile

```
7 # Service Principal Data (Required for Token)
8 $ApplicationID = "<Application/Client ID>"
9 $TenantID = "<Tenant ID>"
10 $CertName = "Data Collect"
11 #
12 # Log Analytics Data (for JSON submission)
13 $DCRImmutableId = "<DCR Immutable ID>"
14 $DCEURI = "<DCE Data Ingestion URL>"
15 $streamName = "<Stream Name from DCR JSON>"
16 #
17 # Script logging
18 $LogFile = "<Log File Path>"
19 #
```

The next block of code are the functions that provide functionality to find the correct certificate and then use it to obtain an access token via OAuth2 after authenticating as the Service Principal.

At the beginning of the script, we use those functions. The script will not run unless a valid certificate is found.

```
157 #
158 # BEGIN SCRIPT
159 #
160 #
161 # Get the first valid certificate that matches $CertName
162 $Cert = Get-ValidCertificate -CertName $CertName
163 # If one was found
164 if ($Cert) {
165     # Get the thumbprint
166     $thumbprint = (Get-Childitem -Path "Cert:\LocalMachine\My" | Where-Object {$_.Subject -Match $CertName}).Thumbprint
167     # BEGIN INVENTORY
168 }
```

Customize the Inventory block to collect the data you want to report via JSON upload. If you are reporting for both Windows and MacIntosh, remember that your JSONs must have identical structures and plan accordingly.

```
167 #
168 # BEGIN INVENTORY
169 #
170 #
171 #
172 # Get the ManagedDeviceID and Name from Intune enrollment
173 try {
174     # If the device is enrolled
175     if (@(Get-Childitem HKLM:SOFTWARE\Microsoft\Enrollments\ -Recurse |
176         # Get the source for the managed device information
```

Use the code example to create the structure we are going to export to JSON.

```
422 #
423 # Compile all the settings in a form we can easily convert to JSON
424 $Inventory = New-Object System.Object
425 $Inventory | Add-Member -MemberType NoteProperty -Name "ManagedDeviceName" -Value $ManagedDeviceName -Force
426 $Inventory | Add-Member -MemberType NoteProperty -Name "AzureADDeviceID" -Value $AzureADDeviceID -Force
427 $Inventory | Add-Member -MemberType NoteProperty -Name "ManagedDeviceID" -Value $ManagedDeviceID -Force
428 $Inventory | Add-Member -MemberType NoteProperty -Name "DefenderState" -Value $DefenderState -Force
429 $Inventory | Add-Member -MemberType NoteProperty -Name "DefenderStart" -Value $DefenderStart -Force
430 $Inventory | Add-Member -MemberType NoteProperty -Name "DefSpySigAge" -Value $Defender_SpySigAge -Force
431 $Inventory | Add-Member -MemberType NoteProperty -Name "DefMissSigAge" -Value $Defender_MissSigAge -Force
432 $Inventory | Add-Member -MemberType NoteProperty -Name "DefAVSigAge" -Value $Defender_AVSigAge -Force
433 $Inventory | Add-Member -MemberType NoteProperty -Name "DefWEngine" -Value $Defender_WEngine -Force
434 $Inventory | Add-Member -MemberType NoteProperty -Name "BitLockerState" -Value $BitLockerState -Force
435 $Inventory | Add-Member -MemberType NoteProperty -Name "BitLockerStart" -Value $BitLockerStart -Force
436 $Inventory | Add-Member -MemberType NoteProperty -Name "BitEncrypted" -Value $BitEncrypted -Force
437 $Inventory | Add-Member -MemberType NoteProperty -Name "BitEncryption" -Value $BitEncryption -Force
438 $Inventory | Add-Member -MemberType NoteProperty -Name "BitProtected" -Value $BitProtected -Force
439 $Inventory | Add-Member -MemberType NoteProperty -Name "BitProtector" -Value $BitProtector -Force
440 foreach ($thiskey in ($NetHash.Keys | Sort-Object)) {
441     $Name = "MAC" + $thiskey.ToString()
442     $Value = $NetHash[$thiskey]
443     $Inventory | Add-Member -MemberType NoteProperty -Name $Name -Value $Value -Force
444 }
445 #
446 # Convert to JSON for sending the data to Log Analytics Workspace
447 $body = $Inventory | ConvertTo-Json
448 #
449 # END INVENTORY
450 #
```

Set a breakpoint on this line of code.

If not, you need to fix something either with your certificate or with your inventory code.

Comment out the highlighted lines of code and remove the breakpoint.

Launch the Azure Portal, search for Log Analytics workspaces.

Click on “Create”.

You may need to create a Resource Group.
If so, click the “Create New” option.

Populate the Name and Region, click “Review + Create”.

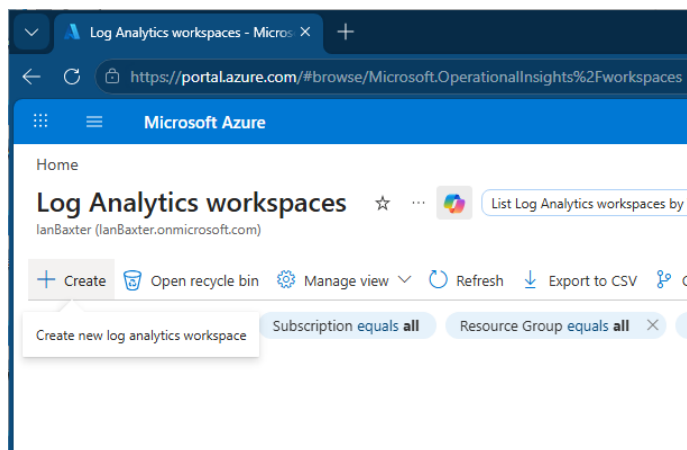
When the validation passes, click “Create”.

Click the “Go to Resource” option.

```

464 # END INVENTORY
465
466 #
467 # Uncomment for DEBUGGING
468 #
469 #
470 # Get the auth token
471 $bearerToken = Get-AuthTokenWithCert -TenantId $TenantID -ClientId $ApplicationID -CertThumbprint $thumbprint
472
473 # Uncomment for DEBUGGING
474 #bearerToken
475
476 # Don't save the whole token to the log, just enough to know we got it
477 $token = $bearerToken.Substring(0, $bearerToken.IndexOf('.')
478 Write-Log -LogFile $LogFile -Message $Message
479
480 # Send the JSON to the Log Analytics Data Ingestion API

```

[illegible]

Project details

Select the subscription to manage deployed resources and costs. Use resource groups like folders to organize and manage all your resources.

Subscription * 



Resource group * 



[Create new](#)

A resource group is a container that holds related resources for an Azure solution.

Name * 



Instance details

Name * 

Region * 

[Review & Create](#) [< Previous](#) [Next > Tags](#)

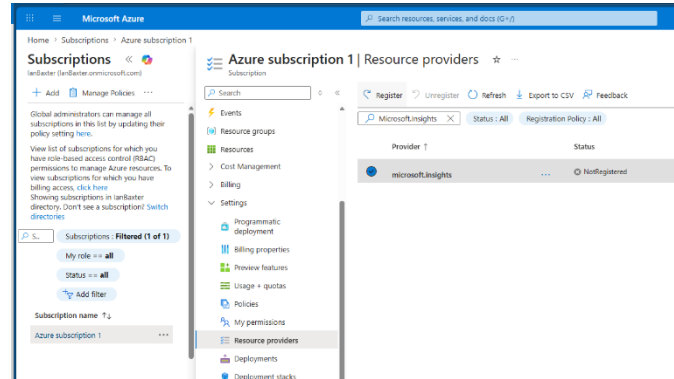
Register Microsoft Insights resource provider

In the Azure Portal, search for “Subscription” and load the applet.

Select the correct subscription. Click on “Resource Providers”, search for “Microsoft.Insights”.

Select it and click the “Register” option.

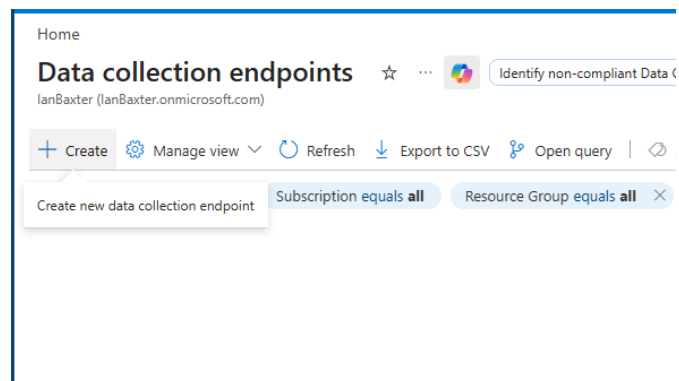
Wait for the notification that the registration has completed.



Create a DCE (Data Collection Endpoint)

Open the Azure Portal, search for “Data Collection Endpoints” and launch the applet.

Click the “Create” option.



Populate the “Endpoint Name”, select the appropriate Subscription, Resource Group and Region.

The Region should be the same as your Tenant and must match the values for the DCR and Log Analytics workspace.

Click “Review and Create.”

Create data collection endpoint

Basics Tags Review + create

Select the subscription to manage deployed resources and costs. Use resource groups like folders to organize and manage all of your resources. [Learn more](#)

Endpoint details

Endpoint Name *	Custom-Compliance ✓
Subscription * ⓘ	Azure subscription 1 ▼
Resource Group * ⓘ	Intune-Reporting ▼ Create new
Region * ⓘ	Canada Central ▼

[Review + create](#) < Previous Next: Tags >

Create the Log Analytics Table and DCR from JSON

Let me caution you here. You need to do this once; do it right and fully understand what you are doing at this step. It seems simple from all the Internet documentation, but it really is not and is poorly documented.

If you make a mistake, you cannot delete the old table and DCR. The data that underlies them can persist for up to 90 days. The data for the table or DCR does not get deleted and will re-populate in any new one you recreate. The only other solution is to create one with a different name.

With the log analytics workspace open, expand the “Settings” and select “Tables”.

Click the “Create” option.

Home > MicrosoftLogAnalyticsOMS | Overview > Custom-Compliance

Custom-Compliance | Tables ☆ ...

Log Analytics workspace

Search

+ Create Create Delete

Filter by name Type: All Plan: All Protection: All

Showing 6 results

Table name ↑↓	Type ↑↓	Plan ↑↓
☐ Alert	Azure table	Analytics
☐ AppCenterError	Azure table	Analytics
☐ ComputerGroup	Azure table	Analytics
☐ InsightsMetrics	Azure table	Analytics
☐ Operation	Azure table	Analytics
☐ Usage	Azure table	Analytics

The table name must be the same as the table you will configure in the script with a few differences. The script uses a stream name which is “Custom-<TableName>_CL”. The “_CL” gets appended to the table name for “Custom Log” in Log Analytics, and the “Custom-” gets added by the DCR to form the data stream name.

Enter a description.

You should not have a DCR yet, so click the “Create a new data collection rule” option.

Home > MicrosoftLogAnalyticsOMS | Overview > Custom-Compliance | Tables

Create a custom log

Table details
Start by adding a name and description for the table you're creating. On the next step, upload a sample of your custom log and adjust the table details to your needs.

Table name * ✓
_CL

Description

Table plan

☒ Analytics ☐ Basic ☐ Auxiliary / Lake

[Learn more about the different table plans by clicking here.](#)

Data collection rule
Data collection rules (DCR) define the data coming into Azure Monitor and specify where that data should be sent or stored. [Learn more](#)

Data collection rule *

Data collection endpoint *

Choose your subscription and resource group, give the DCR a name.

Click “Done”.

Select the Data Collection Endpoint you created earlier, click “Next”.

Create a new data collection rule

Subscription *

Resource group *

[Create new](#)

Region
Canada Central

Name *

[Done](#) [Cancel](#)

Upload the Json data you saved from the script test.

You should see the warning that a transform has been created for TimeGenerated. If your JSON doesn't contain the mandatory TimeGenerated data, a transform will be automatically created for you.

You will also see all the names of values in the JSON you uploaded.

Click “Next” and “Continue”.

The screenshot shows the Microsoft Azure portal interface for creating a custom log. The top navigation bar includes the Microsoft Azure logo and the user's email address. The main heading is "Create a custom log". Below this, there are three tabs: "Basics", "Schema and transformation" (which is selected), and "Review". Under the "Schema and transformation" tab, there is a section for "Upload sample file" and a "Transformation editor" link. A warning message is displayed, stating: "There was no timestamp field found in the sample provided. The transformation was updated to populate TimeGenerated column with the timestamp of data ingestion. Please use the transformation editor to review or update the logic of TimeGenerated column population in the destination table. [Learn more](#)". Below the warning, a table is shown with the following columns: TimeGenerated, ManagedDeviceName, AzureADDeviceID, ManagedDeviceID, DefenderState, and DefenderStart. The table contains one row of data: 2026-06-28T15:18:39..., IanBaxter_Windows_6/..., 6a41bd56-01f6-4b1f-..., c5e23685-75af-4d76-..., Running, and Automatic. At the bottom of the page, there are "Previous" and "Next" buttons.

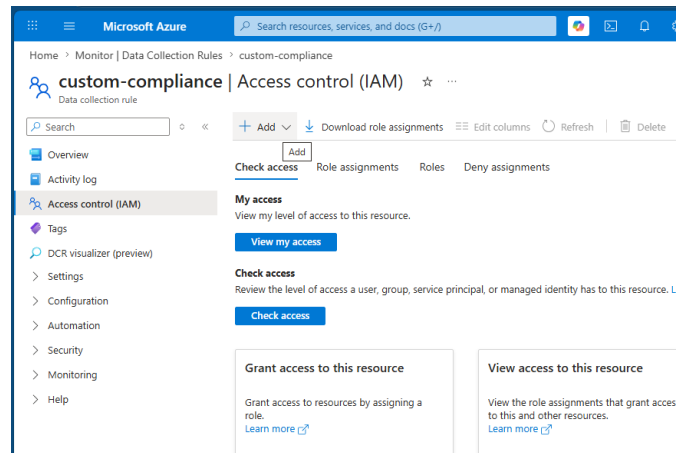
TimeGenerated	ManagedDeviceName	AzureADDeviceID	ManagedDeviceID	DefenderState	DefenderStart
2026-06-28T15:18:39...	IanBaxter_Windows_6/...	6a41bd56-01f6-4b1f-...	c5e23685-75af-4d76-...	Running	Automatic

Permission the Service Principal

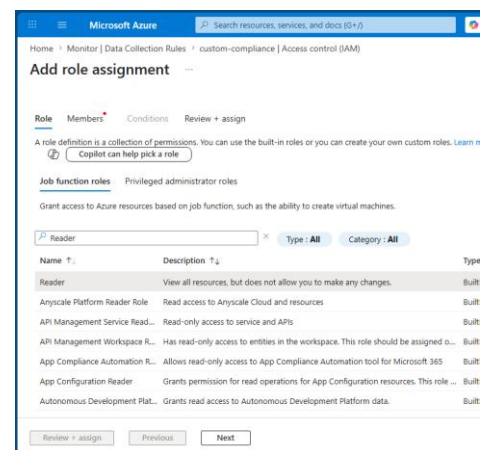
DCR

In the Azure Portal, search for “Data Collection Rule” and launch the applet.

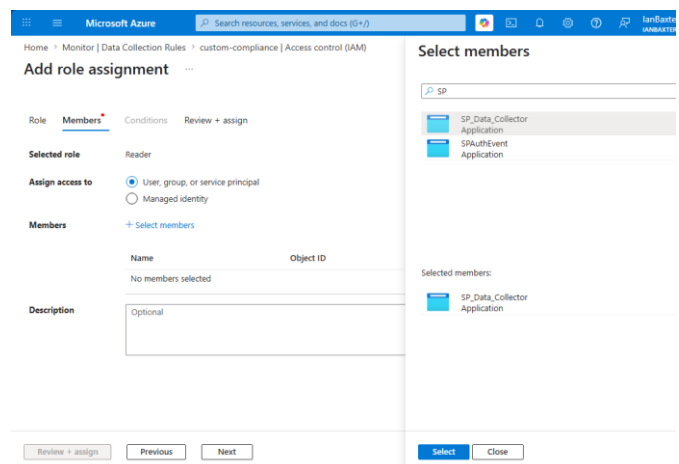
Select “Access Control” and click “Add”.
“Add Role Assignment”.



Search for and select the “Reader” role. Click “Next”.



Click “Add Members”, search for and select the Service Principal account and click the “Select” button.



Click “Review and Assign”.

Repeat these steps to add the “Monitoring Metrics Publisher” role.

Verify the roles have been properly assigned.

Workspace

Search “Log Analytics Workspaces” in the Portal, load the workspace you created.

Add the “Log Analytics Contributor” role to the Service Principal.

Home > Monitor | Data Collection Rules > custom-compliance | Access control (IAM)

Add role assignment

Role **Members** Conditions Review + assign

Selected role Reader

Assign access to ☒ User, group, or service principal ☐ Managed identity

Members + Select members

Name	Object ID	Type
SP_Data_Collector	fb2584b5-98ab-4d7f-86df-622f3dde45fb	App

Description Optional

Review + assign Previous Next

Microsoft Azure Search resources, services, and docs (G47)

Home > Monitor | Data Collection Rules > custom-compliance | Access control (IAM)

Search

Overview Activity log Access control (IAM) Tags DCR visualizer (preview) Settings Configuration Automation Security Monitoring Help

Number of role assignments for this subscription 3 4000

Search by name or email Type: All Role: All Scope: All scopes Group by: Role

All (3) Job function roles (2) Privileged administrator roles (1)

Name	Type	Role	Scope	Condition
Owner (1)				
Ian Baxter ianbaxter@ianba...	User	Owner	Subscription (inheri...	None
Reader (1)				
SP_Data_Collector f68f6252-a149-4...	Service principal	Reader	This resource	None
Monitoring Metrics Publisher (1)				
SP_Data_Collector f68f6252-a149-4...	Service principal	Monitoring Metrics Publ...	This resource	None

Showing 1 - 3 of 3 results.

Add or remove identities by pressing CTRL+click of

Microsoft Azure Search resources, services, and docs (G47)

Home > Log Analytics workspaces > Custom-Compliance | Access control (IAM)

Search

Overview Activity log Access control (IAM) Tags Diagnose and solve problems Logs Resource visualizer Settings Tables Agents Usage and estimated costs Rules Network isolation Identity Linked storage accounts

Check access Role assignments Roles Deny assignments

Number of role assignments for this subscription 4 4000

Search by name or email Type: All Role: All Scope: All scopes Group by: Role

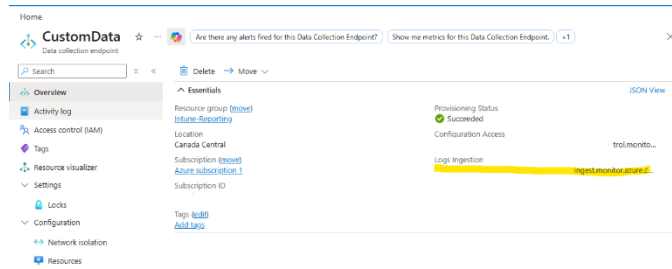
All (2) Job function roles (1) Privileged administrator roles (1)

Name	Type	Role	Scope	Condition
Owner (1)				
Ian Baxter ianbaxter@ianba...	User	Owner	Subscription (inheri...	None
Log Analytics Contributor (1)				
SP_Data_Collector f68f6252-a149-4...	Service principal	Log Analytics Contributor	This resource	None

Showing 1 - 2 of 2 results.

Update the script for the DCE and DCR and test

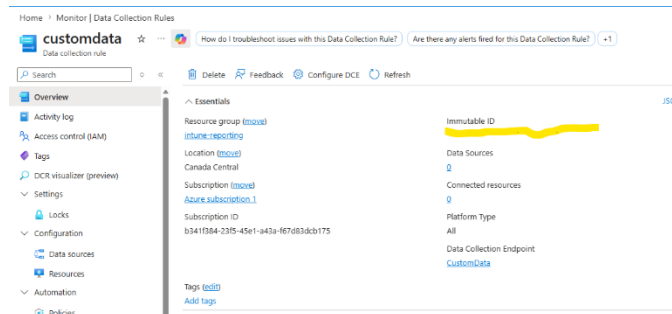
Copy the Logs Ingestion URL from the DCE Overview.



Paste the URL into the configuration block in the script.

```
11 #
12 # Log Analytics Data (for JSON submission)
13 $DcrImmutableId = "<DCR Immutable ID>"
14 $DceURI = "<DCE Data Ingestion URL>"
15 $streamName = "<Stream Name from DCR JSON>"
16 #
17 # Script loading
```

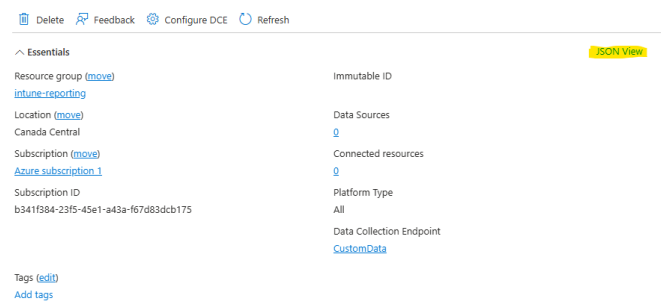
Copy the DCR Immutable ID from the DCR Overview.



Paste the ID into the configuration block of the script.

```
12 # Log Analytics Data (for JSON submission)
13 $DcrImmutableId = "<DCR Immutable ID>"
14 $DceURI = "<DCE Data Ingestion URL>"
15 $streamName = "<Stream Name from DCR JSON>"
16 #
```

In the DCR Overview, click the "JSON View" link.



Look for this section of the code and copy the stream name.

```
9 |
10 |
11 | "dataCollectionEndpointId": "/subscriptions/b341f384-23f5-45e1-a43a-f67d8
12 | "streamDeclarations": {
13 |   "Custom-CustomData-Ck": {
14 |     "columns": [
15 |       {
16 |         "name": "TimeGenerated",
17 |         "type": "datetime"
18 |       },
19 |     ],
20 |   },
21 | }
```

Paste the stream name into the correct line of the script.

Make sure you have a log file path specified. Given Intune's console ability to collect logs remotely, this can be very useful.

Besides the log, Diagnostic Settings in a DCR can also help with debugging.

Load the DCR, select "Diagnostic Settings" in the left menu and click "Add Diagnostic Setting..."

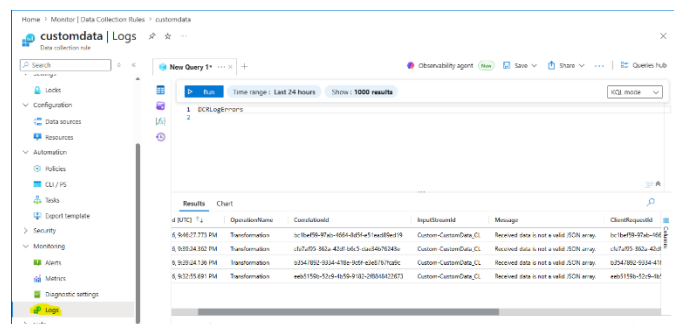
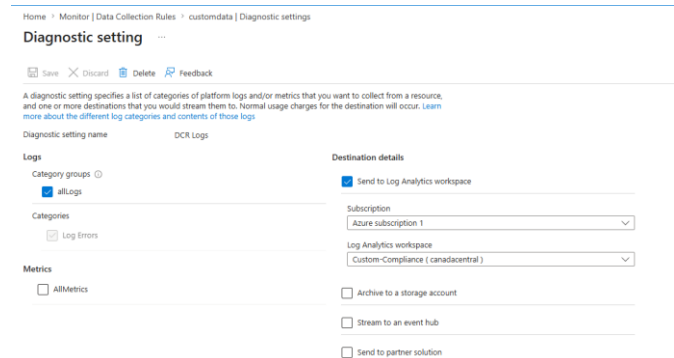
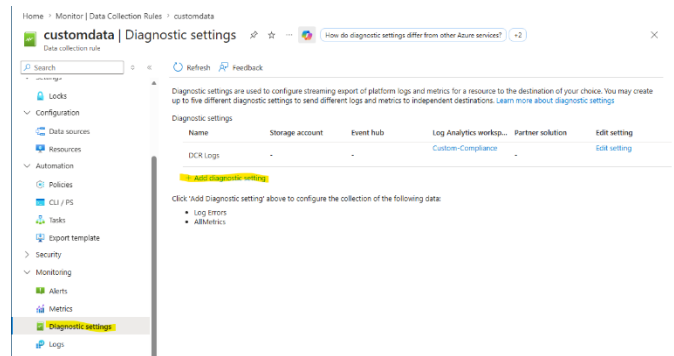
Configuring these options will send all the DCR logs to the Log Analytics workspace.

Selecting Logs in the left pane and clicking the "Run" button will show you any found issues.

If the configuration is correct, you can run the script and get a success message.

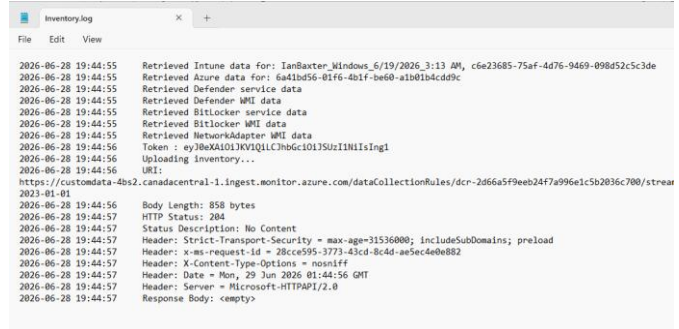
```
10 # Paste the stream name into the correct line of the script
11 #
12 # Log Analytics Data (for JSON submission)
13 $DcrImmutableId = "<DCR Immutable ID>"
14 $DceURI = "<DCE Data Ingestion URL>"
15 $streamName = "<Stream Name from DCR JSON>"
16 #
17 # Script logging
```

```
15 $streamName = "<Stream Name from DCR JSON>"
16 #
17 # Script logging
18 $LogFile = "<Log File Path>"
19 #
20 # If a log file has been specified and it already exists, d
21 if (-not [string]::IsNullOrEmpty($LogFile) -and (Test-
```

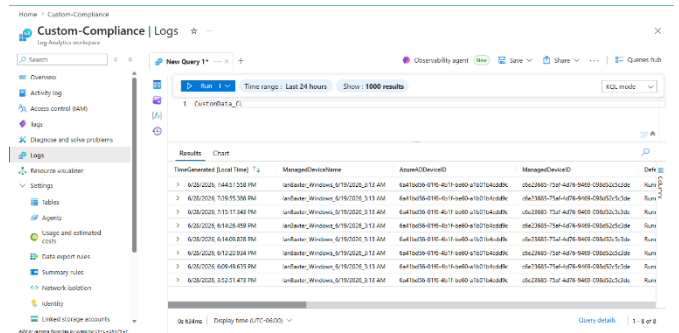


```
PS C:\WINDOWS\system32> E:\My Drive\Repo\Intune Lab\Projects\Custom Reporting\SP\Custom_Inventory.ps1
InventoryDate:28-06-19:39 DeviceInventory:OK (204)
PS C:\WINDOWS\system32> |
```

If not, the logs should show you where you went wrong.



If it went right, the data can be viewed in the Log Analytics table.



Once everything is working you can remove the Diagnostics from the DCR, they should not be needed in the future if you still have the clients logging properly.

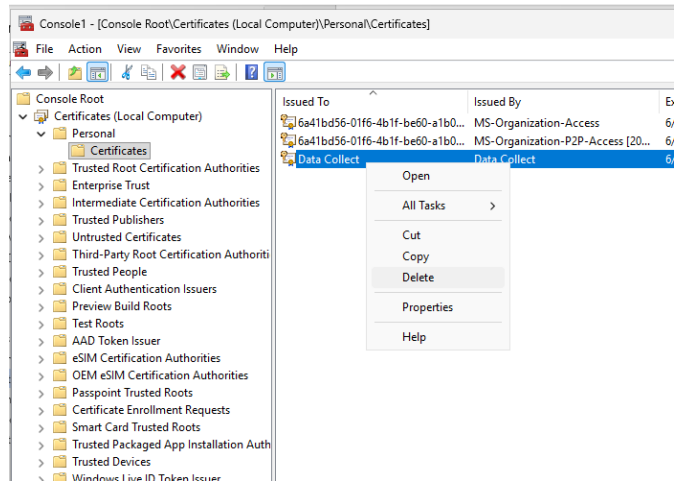
Remove the manually deployed Certificate

Using Windows search, find “CMD”, right-click and choose “Run as Administrator”.

Load MMC and the Certificates snap in using the same responses as you did earlier.

Locate the certificate in the “Personal” folder, right-click on it and choose “Delete”.

Confirm the delete and check if it remains listed.



Deploy the Private Key to all devices

This certificate is self-signed and needs a creative solution, deployment as a Win32 app.

You need to use PowerShell scripts to install, uninstall and detect the certificate installation. You will need to have the certificate thumbprint and password. The resulting packages (.intunewin) are encrypted twice, so they should be secure.

Install.ps1

Code:

```
# Path to the PFX inside the win32 app package
$PfxPath = "$PSScriptRoot\<CERTFILE>.pfx"

# PFX password (replace with your actual password)
$Password = "<PASSWORD>" | ConvertTo-SecureString -AsPlainText -Force

Write-Host "Importing PFX certificate into LocalMachine\My..."

Import-PfxCertificate `
    -FilePath $PfxPath `
    -CertStoreLocation Cert:\LocalMachine\My `
    -Password $Password

Write-Host "Certificate import completed."
```

Make sure to replace the "<CERTFILE>" and "<PASSWORD>" blocks.

Uninstall.ps1

Code:

```
$Thumbprint = "<THUMBPRINT>"
$Store = Get-ChildItem -Path Cert:\LocalMachine\My

$Cert = $Store | Where-Object { $_.Thumbprint -eq $Thumbprint }

if ($Cert) {
    Write-Host "Removing certificate with thumbprint $Thumbprint..."
    $Store.Remove($Cert)
    Write-Host "Certificate removed."
} else {
    Write-Host "Certificate not found. Nothing to remove."
}
```

Make sure to replace the "<THUMBPRINT>" block.

Detect.ps1

Code:

```
$Thumbprint = "<THUMBPRINT>"
$Store = Get-ChildItem -Path Cert:\LocalMachine\My

$Cert = $Store | Where-Object { $_.Thumbprint -eq $Thumbprint }

if ($Cert) {
    Write-Host "Detected"
    exit 0
} else {
    Write-Host "Not detected"
    exit 1
}
```

Create a folder, put the PowerShell scripts and certificate file in the folder. Use the IntuneWinAppUtil executable to create the package to upload.

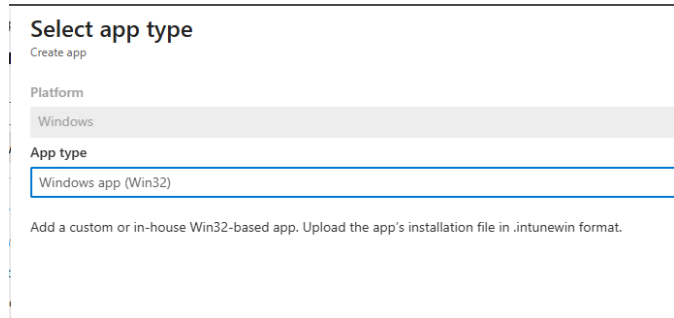
Go to the Intune Console and create a Windows App.

Set the App Type to “Windows app (Win32)”.

Select your package file and click “OK”.

Populate the next form, you might want to enforce strict versioning so you can use app supersedence with future updates.

Click “Next”.



Select app type

Create app

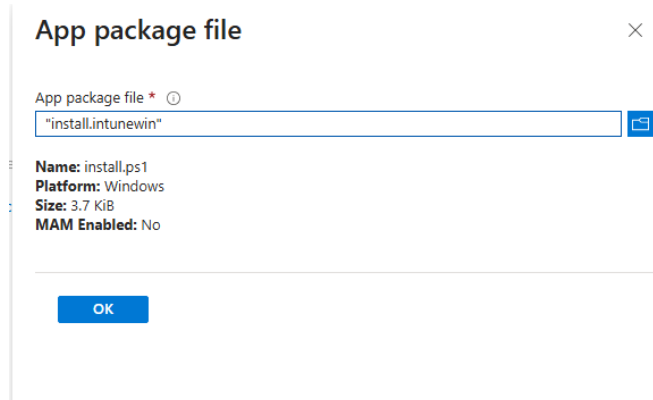
Platform

Windows

App type

Windows app (Win32)

Add a custom or in-house Win32-based app. Upload the app's installation file in .intunewin format.



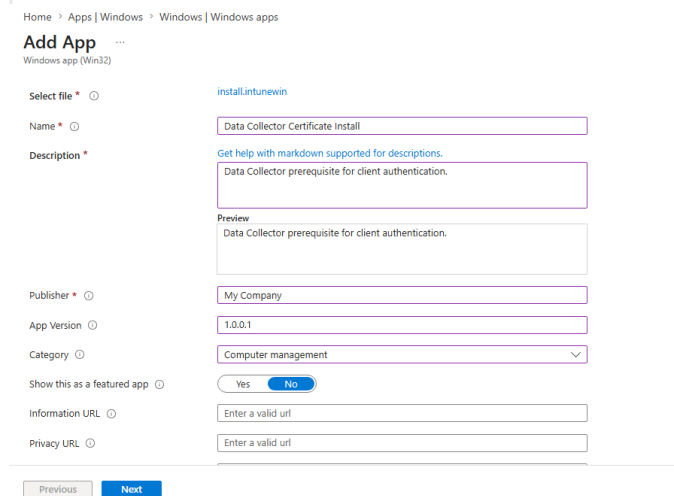
App package file

App package file *

install.intunewin

Name: install.ps1
Platform: Windows
Size: 3.7 KiB
MAM Enabled: No

OK



Home > Apps | Windows > Windows | Windows apps

Add App

Windows app (Win32)

Select file *

install.intunewin

Name *

Data Collector Certificate Install

Description *

Data Collector prerequisite for client authentication.

Preview

Data Collector prerequisite for client authentication.

Publisher *

My Company

App Version *

1.0.0.1

Category *

Computer management

Show this as a featured app

Yes No

Information URL

Enter a valid url

Privacy URL

Enter a valid url

Previous Next

Populate the next form

Install command:

powershell.exe -ExecutionPolicy Bypass -File Install.ps1

Uninstall command:

powershell.exe -ExecutionPolicy Bypass -File Uninstall.ps1

Users should not be able to uninstall this.

It needs to be installed in the SYSTEM context.

DO NOT CLICK NEXT! SCROLL DOWN.

Populate the expected return codes.

Click “Next”.

Set the installation requirements.

Click “Next”.

Home > Apps | Windows > Windows | Windows apps

Add App

Windows app (Win32)

App information

Program

Requirements

Detection rules

Dependencies

Specify app installation and Uninstallation settings, including whether to use a script or command line, time limits, restart behavior, and return codes.

Installer type

Command line

Install command

powershell.exe -ExecutionPolicy Bypass -File Install.ps1

Uninstaller type

Command line

Uninstall command

powershell.exe -ExecutionPolicy Bypass -File Uninstall.ps1

Installation time required (mins)

60

Allow available uninstall

No

Install behavior

System

Device restart behavior

App install may force a device restart

Specify return codes to indicate post-installation behavior:

Return code

Code type

Previous

Next

Specify return codes to indicate post-installation behavior:

Return code

Code type

0

Success

1

Failed

2

Retry

3

Hard reboot

-2147024894

Failed

-2147023239

Failed

-2145124329

Retry

+ Add

Home > Apps | Windows > Windows | Windows apps

Add App

Windows app (Win32)

App information

Program

Requirements

Detection rules

Dependencies

Specify the requirements that devices must meet before the app is installed:

Check operating system architecture

No. Allow this app to be installed on all systems.

Minimum operating system

Windows 11 21H2

Disk space required (MB)

Physical memory required (MB)

Minimum number of logical processors required

Minimum CPU speed required (MHz)

Configure additional requirement rules

Type

Path/Script

No requirements are specified.

Previous

Next

Select “Use a custom detection script” for the “Rules format” and upload the Detect.ps1 script.

Click “Next”.

Home > Apps | Windows > Windows | Windows apps

Add App

Windows app (Win32)

✓ App information ✓ Program ✓ Requirements **1 Detection rules** ① Dependencies ② Supersedence

Configure app specific rules used to detect the presence of the app.

Rules format * ① Use a custom detection script

Script file ① Detect.ps1

Script content

```
$Thumbprint = "1c2f63a9eb10553a9795ae75be91bbdae9bb29f6"
$Store = Get-Childitem -Path Cert:\LocalMachine\My

$Cert = $Store | Where-Object { $_.Thumbprint -eq $Thumbprint }

if ($Cert) {
    Write-Host "Detected"
}
```

Run script as 32-bit process on 64-bit clients ① Yes **No**

Enforce script signature check and run script silently ① Yes **No**

Previous Next

Click “Next” in the “Dependencies” and “Supersedence” forms.

This should be a Required deployment for All Devices.

Click “Next” when done.

Home > Apps | Windows > Windows | Windows apps

Add App

Windows app (Win32)

✓ Program ✓ Requirements ✓ Detection rules ✓ Dependencies ✓ Supersedence **1 Assignments** ...

Any Win32 app deployed using Intune will not be automatically removed from the device when the device is retired. The app and the data it contains will remain on the device. If the app is not removed prior to retiring the device, the end user will need to take explicit action on the device to remove the app.

Required ①

Group mode	Group	Status	Filter mode	Filter	End user notifications
Included	All devices	Active	None	None	Show all toast notifications

+ Add group ① + Add all users ① + Add all devices ①

Available for enrolled devices ①

Group mode	Group	Status	Filter mode	Filter	End user notifications
No assignments					

+ Add group ① + Add all users ① + Add all devices ①

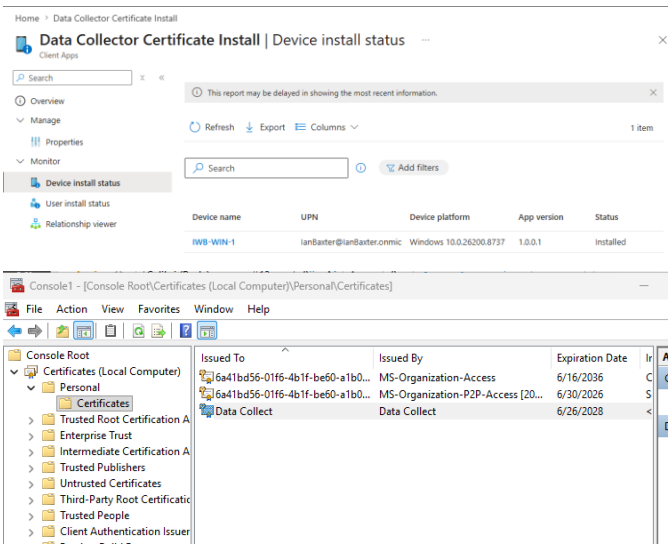
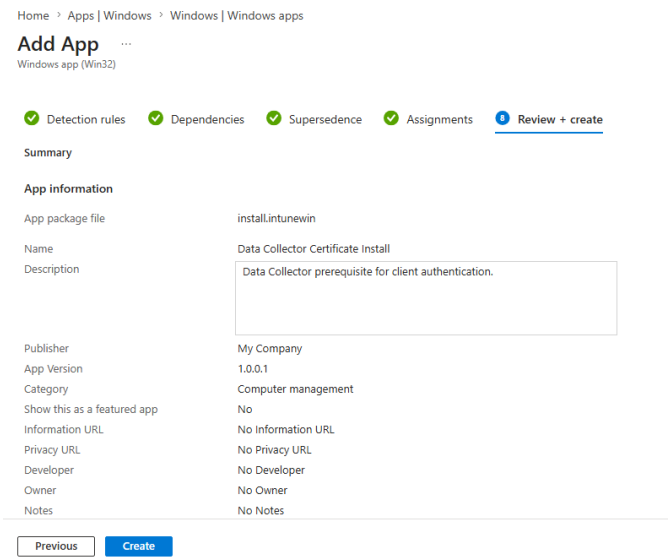
Previous Next

Review and when you are sure it is correct, click “Create”.

Wait for it to deploy and check for any issues. It should be deployed to and be confirmed on your test device.

Verified deployed.

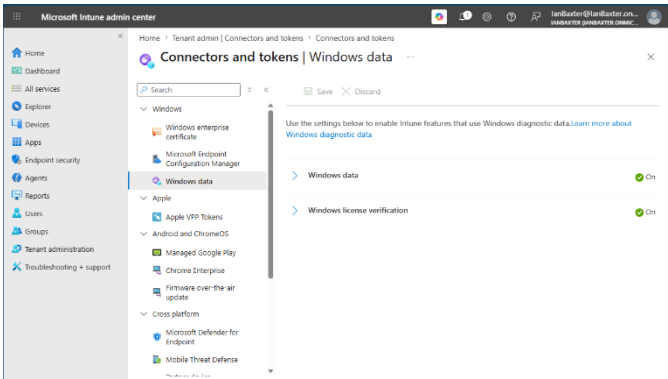
Verified Installed.



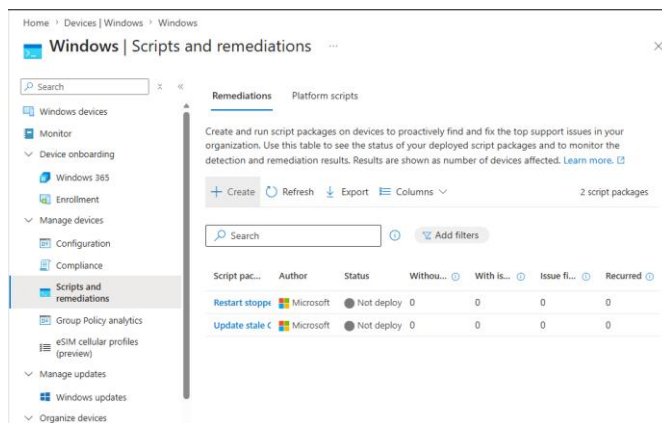
Create the Custom Compliance Policy and Deploy

To use Remediations, you need to ensure Windows License Validation is enabled.

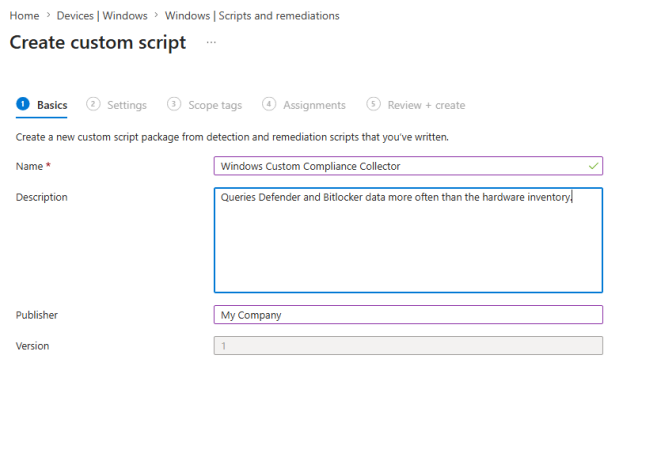
Go the “Tenant Administration”, “Connector and Tokens”, “Windows Data” and verify it is enabled.



Under “Devices”, “Windows”, “Scripts and Remediations” select the “Remediations” option and click “Create”.

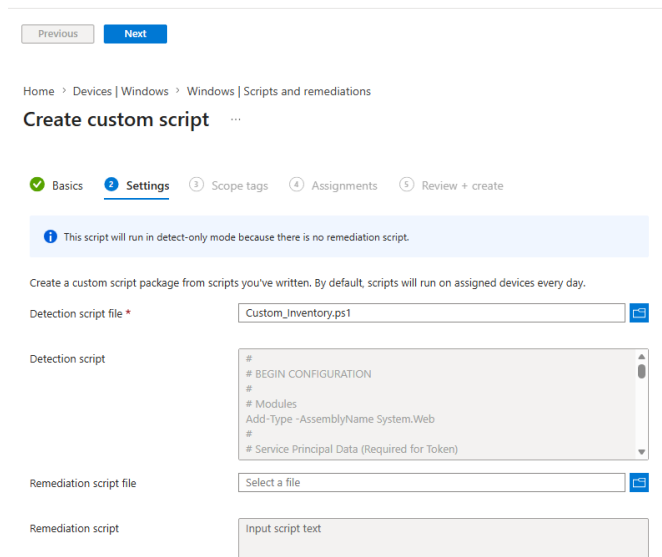


Give the script a name and description.
The publisher can be whatever you decide.

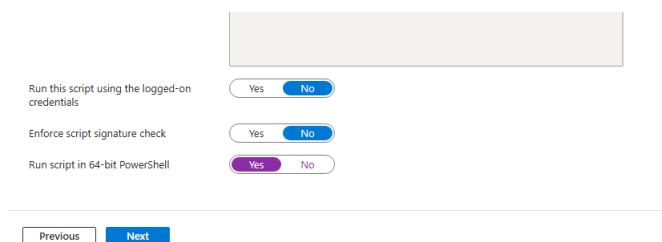


Click “Next”.

Upload the Custom Inventory script and
scroll down.



Ensure the script does not run as the user
and make sure it is running in 64-bit
PowerShell.



Click “Next”.

The Default scope tag is good enough for testing.

Click “Next”.

Assign the script to “All Devices”.

Note that the Schedule for the script is “Daily”. Click on the blue text to change the script frequency.

You need to consider how often this script runs. I prefer to run it every 6 hours. In a global organization, this timing ensures that at least once day when the device is being used, it will report.

It’s also very handy during testing to collect data more often.

Click “Apply” and “Next”.

Home > Devices | Windows > Windows | Scripts and remediations

Create custom script

1 Basics 2 Settings 3 Scope tags 4 Assignments 5 Review + create

Select one or more groups to assign the script package.

Scope tags

Default

+ Select scope tags

Home > Devices | Windows > Windows | Scripts and remediations

Create custom script

1 Basics 2 Settings 3 Scope tags 4 Assignments 5 Review + create

Select one or more groups to assign the script package.

Included groups

Assign to All devices

Selected groups	Schedule	Filter	Filter mode
All devices	Daily	None	None

Excluded groups

Include or exclude either device groups or user groups. Don't mix user and device groups across include and exclude assignments.

Selected groups

No groups selected

Previous Next

Home > Devices | Windows > Windows | Scripts and remediations

Create custom script

1 Basics 2 Settings 3 Scope tags 4 Assignments 5 Review + create

Select one or more groups to assign the script package.

Included groups

Assign to All devices

Selected groups	Schedule	Filter	Filter mode
All devices	Daily	None	None

Excluded groups

Include or exclude either device groups or user groups. Don't mix user and device groups across include and exclude assignments.

Selected groups

No groups selected

Previous Next

Edit settings

Schedule

Create a schedule for this script to run on devices in group: All devices.

Frequency

Hourly

Repeats every *

6 hours

Apply Reset

Confirm your settings and click “Create”.

Home > Devices | Windows > Windows | Scripts and remediations

Create custom script

✓ Basics

✓ Settings

✓ Scope tags

✓ Assignments

1 Review + create

Summary

Basics

Name	Windows Custom Compliance Collector
Description	Queries Defender and Bitlocker data more often than the hardware inventory.
Publisher	My Company
Version	No Version

Settings

Detection script	Yes
Remediation script	No
Run this script using the logged-on credentials	No
Enforce script signature check	No
Run script in 64-bit PowerShell	Yes

Scope tags

Previous

Create

Going back to the Scripts and Remediations shows it is active and ready to go.

Home > Devices | Windows > Windows

Windows | Scripts and remediations

Search

Windows devices

Monitor

Device onboarding

Windows 365

Enrollment

Manage devices

Configuration

Compliance

Scripts and remediations

Group Policy analytics

eSIM cellular profiles (preview)

Manage updates

Windows updates

On-premise devices

Remediations

Platform scripts

Create and run script packages on devices to proactively find and fix the top support issues in your organization. Use this table to see the status of your deployed script packages and to monitor the detection and remediation results. Results are shown as number of devices affected. [Learn more.](#)

Create

Refresh

Export

Columns

3 script packages

Search

Add filters

Script pac...	Author	Status	Without...	With is...	Issue fl...	Recurred
Restart stoppx	Microsoft	Not deploy	0	0	0	0
Windows Cust	My Company	Active	0	0	0	0
Update stale c	Microsoft	Not deploy	0	0	0	0

The script finally deployed and ran without issue.

Home > Devices | Windows > Windows

Windows | Scripts and remediations

Search

Windows devices

Monitor

Device onboarding

Windows 365

Enrollment

Manage devices

Configuration

Compliance

Scripts and remediations

Group Policy analytics

eSIM cellular profiles (preview)

Manage updates

Remediations

Platform scripts

Create and run script packages on devices to proactively find and fix the top support issues in your organization. Use this table to see the status of your deployed script packages and to monitor the detection and remediation results. Results are shown as number of devices affected. [Learn more.](#)

Create

Refresh

Export

Columns

3 script packages

Search

Add filters

Script pac...	Author	Status	Without...	With is...	Issue fl...	Recurred
Restart stoppx	Microsoft	Not deploy	0	0	0	0
Windows Cust	My Company	Active	1	0	0	0
Update stale c	Microsoft	Not deploy	0	0	0	0

Turning on the Pre-Remediation Detection Output shows us the result from the script.

Home > Devices | Windows > Windows | Scripts and remediations > Windows Custom Compliance Collector

Windows Custom Compliance Collector | Device status

Proactive remediations

Overview

Manage

Properties

Monitor

Device status

Refresh

Export

Columns

Search

Device name	User	35 version	Last run	Pre remediation detection output
WS2-WIN-1	larSa		6/28/2024, 02:00:23 PM	InventoryData29 06 13:58...Review

✓ Last run

✓ OS version

Pre-remediation detection error

Post remediation detection output

Pre-remediation detection error

Pre-remediation detection output

Remediation error

✓ Remediation status

✓ User name

Clicking “Review” allows us to read the full output.

Scripts and remediations > Windows Custom Compliance Collector

ompliance Collector | Device status

Refresh

Export

Columns

Search

Add filters

Device name	User name	Detection sta...	Remediation
IWB-WIN-1	lan@baxter@lan...	Without issue	Not run

Pre-remediation detection output

Windows Custom Compliance Collector

InventoryDate:29-06 13:58 DeviceInventory:OK (204)

Verify Data is being collected

Using a KQL query in Log Analytics shows that the data is being collected.

Compliance

Custom-Compliance | Logs

Log Analytics workspace

Search

New Query 1*

Overview

Activity log

Access control (IAM)

Tags

Diagnose and solve problems

Logs

Resource visualizer

Settings

Tables

Agents

Usage and estimated costs

Data export rules

Summary rules

Network isolation

Identity

Observability agent

Save

Share

Queries hub

Run

Time range: Last 4 hours

Show: 1000 results

KQL mode

1 CustomData_CL

Results

Chart

TimeGenerated [UTC]	ManagedDeviceName	AcmeADDeviceID	ManagedDeviceID
> 6/29/2026, 7:58:24.875 PM	lan@baxter_Windows_6/19/2026...	6a41ba56-0186-4b11-be60-a7b...	cbe23885-73af-4d76-9469-098...